

DSA

ALGORITHMS

Python Edition

Theory · Algorithm · Code · Complexity

Sorting · Searching · Dynamic Programming · Graphs Greedy · Backtracking ·
Mathematical · Trees

Table of Contents

1. Sorting Algorithms

- 1.1 Bubble Sort
- 1.2 Selection Sort
- 1.3 Insertion Sort
- 1.4 Merge Sort
- 1.5 Quick Sort
- 1.6 Heap Sort
- 1.7 Counting Sort

2. Searching Algorithms

- 2.1 Linear Search
- 2.2 Binary Search
- 2.3 Jump Search
- 2.4 Interpolation Search
- 2.5 Exponential Search
- 2.6 Ternary Search

3. Dynamic Programming Algorithms

- 3.1 Fibonacci Sequence
- 3.2 0/1 Knapsack Problem
- 3.3 Longest Common Subsequence (LCS)
- 3.4 Longest Increasing Subsequence (LIS)
- 3.5 Matrix Chain Multiplication
- 3.6 Edit Distance (Levenshtein)

4. Graph Algorithms

- 4.1 Depth First Search (DFS)
- 4.2 Breadth First Search (BFS)
- 4.3 Dijkstra's Algorithm
- 4.4 Bellman-Ford Algorithm
- 4.5 Floyd-Warshall Algorithm

- 4.6 Kruskal's Algorithm (MST)
- 4.7 Prim's Algorithm (MST)
- 4.8 A* Search Algorithm
- 4.9 Tarjan's Algorithm (SCC)
- 4.10 Kosaraju's Algorithm (SCC)
- 4.11 Cycle Detection (Directed & Undirected)
- 4.12 Topological Sorting
- 4.13 Union-Find (Disjoint Set Union)

5. Greedy Algorithms

- 5.1 Activity Selection Problem

6. Backtracking Algorithms

- 6.1 N-Queens Problem
- 6.2 Sudoku Solver

7. Mathematical Algorithms

- 7.1 Euclidean Algorithm (GCD)
- 7.2 Sieve of Eratosthenes
- 7.3 Modular Exponentiation

8. Bit Manipulation Algorithms

- 8.1 Counting Set Bits
- 8.2 Finding the Only Non-Duplicated Element

9. Tree Algorithms

- 9.1 Tree Traversals
- 9.2 Binary Search Tree (BST)
- 9.3 Lowest Common Ancestor (LCA)
- 9.4 AVL Tree (Self-Balancing BST)

1. Sorting Algorithms

1.1. Bubble Sort

Theory

Bubble Sort is a simple comparison-based sorting algorithm. It repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. Smaller elements 'bubble' to the top while larger ones sink.

Algorithm

1. Start from the first element, compare it with the next.
2. If the current element is greater than the next element, swap them.
3. Move to the next element and repeat for all elements.
4. Continue until no swaps are needed (list is sorted).

Code (Python)

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        swapped = False
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break

arr = [5, 1, 4, 2, 8]
bubble_sort(arr)
print(arr) # [1, 2, 4, 5, 8]
```

Time & Space Complexity

Complexity	Value
Best Case	$O(n)$ — already sorted
Average Case	$O(n^2)$
Worst Case	$O(n^2)$
Space	$O(1)$

1.2. Selection Sort

Theory

Selection Sort repeatedly selects the minimum element from the unsorted portion and places it at the beginning. It requires $O(n^2)$ comparisons regardless of input.

Algorithm

1. Start at position 0 and find the minimum in the full array.
2. Swap the minimum with the element at position 0.
3. Move to position 1 and repeat for the remaining unsorted portion.
4. Continue until the array is fully sorted.

Code (Python)

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]

arr = [64, 25, 12, 22, 11]
selection_sort(arr)
print(arr) # [11, 12, 22, 25, 64]
```

Time & Space Complexity

Complexity	Value
Best Case	$O(n^2)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$
Space	$O(1)$

1.3. Insertion Sort

Theory

Insertion Sort builds the sorted array one element at a time by inserting each element into its correct position among already-sorted elements. Efficient for small or nearly sorted datasets.

Algorithm

1. Consider the second element; compare it with the previous elements.
2. Shift elements greater than the key one position right.
3. Insert the key at the correct position.
4. Repeat for every remaining element.

Code (Python)

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key

arr = [12, 11, 13, 5, 6]
insertion_sort(arr)
print(arr) # [5, 6, 11, 12, 13]
```

Time & Space Complexity

Complexity	Value
Best Case	$O(n)$ — already sorted
Average Case	$O(n^2)$
Worst Case	$O(n^2)$
Space	$O(1)$

1.4. Merge Sort

Theory

Merge Sort is a Divide-and-Conquer algorithm. It splits the array in half, recursively sorts each half, then merges the two sorted halves. Stable and predictably $O(n \log n)$ in all cases.

Algorithm

1. Divide the array into two halves.
2. Recursively sort each half.
3. Merge the two sorted halves back into one sorted array.

Code (Python)

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result, i, j = [], 0, 0
```

```

while i < len(left) and j < len(right):
    if left[i] <= right[j]:
        result.append(left[i]); i += 1
    else:
        result.append(right[j]); j += 1
result.extend(left[i:])
result.extend(right[j:])
return result

arr = [12, 11, 13, 5, 6, 7]
print(merge_sort(arr)) # [5, 6, 7, 11, 12, 13]

```

Time & Space Complexity

Complexity	Value
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n \log n)$
Space	$O(n)$

1.5. Quick Sort

Theory

Quick Sort picks a pivot element and partitions the array so elements smaller than the pivot go left and larger go right, then recursively sorts each side. Fastest in practice with $O(n \log n)$ average time.

Algorithm

1. Choose a pivot (here: last element).
2. Partition: move elements smaller than pivot to the left.
3. Recursively apply to left and right sub-arrays.

Code (Python)

```

def quick_sort(arr, low=0, high=None):
    if high is None:
        high = len(arr) - 1
    if low < high:
        pi = partition(arr, low, high)
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)

def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:

```

```

        i += 1
        arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1

arr = [10, 7, 8, 9, 1, 5]
quick_sort(arr)
print(arr) # [1, 5, 7, 8, 9, 10]

```

Time & Space Complexity

Complexity	Value
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n^2)$ — unbalanced pivot
Space	$O(\log n)$

1.6. Heap Sort

Theory

Heap Sort converts the array into a max-heap, then repeatedly extracts the maximum element and places it at the end, shrinking the heap each time. In-place with guaranteed $O(n \log n)$.

Algorithm

1. Build a max-heap from the array.
2. Swap the root (max) with the last element and reduce heap size.
3. Heapify the root to restore the heap property.
4. Repeat until the heap is empty.

Code (Python)

```

def heap_sort(arr):
    n = len(arr)
    # Build max-heap
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
    # Extract elements
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        heapify(arr, i, 0)

def heapify(arr, n, i):
    largest = i
    l, r = 2*i + 1, 2*i + 2
    if l < n and arr[l] > arr[largest]:

```



```

        largest = l
    if r < n and arr[r] > arr[largest]:
        largest = r
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

arr = [12, 11, 13, 5, 6, 7]
heap_sort(arr)
print(arr) # [5, 6, 7, 11, 12, 13]

```

Time & Space Complexity

Complexity	Value
Best Case	$O(n \log n)$
Average Case	$O(n \log n)$
Worst Case	$O(n \log n)$
Space	$O(1)$

1.7. Counting Sort

Theory

Counting Sort is a non-comparison-based algorithm that counts occurrences of each element and uses prefix sums to place elements in the correct position. Works efficiently when the value range k is not much larger than n .

Algorithm

1. Find the maximum value k in the array.
2. Create a count array of size $k+1$ and count occurrences.
3. Compute prefix sums to determine positions.
4. Build the output array using the count array.

Code (Python)

```

def counting_sort(arr):
    if not arr:
        return arr
    k = max(arr)
    count = [0] * (k + 1)
    for x in arr:
        count[x] += 1
    # Prefix sums
    for i in range(1, k + 1):
        count[i] += count[i - 1]
    output = [0] * len(arr)

```

```
for x in reversed(arr):
    count[x] -= 1
    output[count[x]] = x
return output

arr = [4, 2, 2, 8, 3, 3, 1]
print(counting_sort(arr)) # [1, 2, 2, 3, 3, 4, 8]
```

Time & Space Complexity

Complexity	Value
Best Case	$O(n + k)$
Average Case	$O(n + k)$
Worst Case	$O(n + k)$
Space	$O(k)$

2. Searching Algorithms

2.1. Linear Search

Theory

Linear Search checks each element one by one until the target is found or the list is exhausted. Simple but $O(n)$ — suitable for small or unsorted collections.

Algorithm

1. Start from the first element.
2. Compare with target; return index if matched.
3. Advance to next element and repeat.
4. Return -1 if the end is reached without a match.

Code (Python)

```
def linear_search(arr, target):
    for i, val in enumerate(arr):
        if val == target:
            return i
    return -1

arr = [10, 23, 45, 70, 11, 15]
print(linear_search(arr, 70)) # 3
print(linear_search(arr, 99)) # -1
```

Time & Space Complexity

Complexity	Value
Best Case	$O(1)$ — target is first
Average Case	$O(n)$
Worst Case	$O(n)$

2.2. Binary Search

Theory

Binary Search works on sorted arrays by halving the search range each step. It is far more efficient than linear search for large sorted datasets.

Algorithm

1. Set low=0 and high=len-1.

2. Compute $\text{mid} = (\text{low} + \text{high}) // 2$.
3. If $\text{arr}[\text{mid}] == \text{target}$, return mid .
4. If $\text{target} < \text{arr}[\text{mid}]$, set $\text{high} = \text{mid} - 1$; else set $\text{low} = \text{mid} + 1$.
5. Repeat until $\text{low} > \text{high}$.

Code (Python)

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

arr = [1, 3, 5, 7, 9, 11]
print(binary_search(arr, 7)) # 3
print(binary_search(arr, 4)) # -1
```

Time & Space Complexity

Complexity	Value
Best Case	$O(1)$ — target at mid
Average Case	$O(\log n)$
Worst Case	$O(\log n)$

2.3. Jump Search

Theory

Jump Search works on sorted arrays by jumping ahead by $\sqrt{n}$ steps, then performing a linear search within the found block.

Algorithm

1. Jump ahead by $\text{block} = \text{int}(\sqrt{n})$ steps.
2. Stop when $\text{arr}[\min(\text{step}, n)-1] \geq \text{target}$.
3. Perform linear search within the identified block.
4. Return the index if found, else -1.

Code (Python)

```
import math
```

```
def jump_search(arr, target):
    n = len(arr)
    step = int(math.sqrt(n))
    prev = 0
    while arr[min(step, n) - 1] < target:
        prev = step
        step += int(math.sqrt(n))
        if prev >= n:
            return -1
    for i in range(prev, min(step, n)):
        if arr[i] == target:
            return i
    return -1

arr = list(range(11)) # [0..10]
print(jump_search(arr, 8)) # 8
```

Time & Space Complexity

Complexity	Value
Best Case	$O(1)$
Average Case	$O(\sqrt{n})$
Worst Case	$O(\sqrt{n})$

2.4. Interpolation Search

Theory

Interpolation Search improves on Binary Search for uniformly distributed data by using interpolation to estimate the likely position of the target.

Algorithm

1. Compute $pos = low + (target - arr[low]) * (high - low) // (arr[high] - arr[low])$.
2. If $arr[pos] == target$, return pos.
3. If $target < arr[pos]$, search left; else search right.
4. Repeat until found or range is invalid.

Code (Python)

```
def interpolation_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high and arr[low] <= target <= arr[high]:
        if low == high:
            return low if arr[low] == target else -1
        pos = low + (target - arr[low]) * (high - low) // (arr[high] - arr[low])
        if arr[pos] == target:
            return pos
```

```

        elif arr[pos] < target:
            low = pos + 1
        else:
            high = pos - 1
    return -1

arr = [10, 12, 13, 16, 18, 19, 21, 23, 24, 33, 35]
print(interpolation_search(arr, 18)) # 4

```

Time & Space Complexity

Complexity	Value
Best Case	O(1)
Average Case	O(log log n)
Worst Case	O(n)

2.5. Exponential Search

Theory

Exponential Search finds the range where the target may exist by doubling the index, then applies Binary Search within that range. Useful when the array size is unknown.

Algorithm

1. Start at index 1 and double until $\text{arr}[i] > \text{target}$ or end of array.
2. Apply Binary Search in the range $[i//2, \min(i, n-1)]$.

Code (Python)

```

def exponential_search(arr, target):
    if not arr:
        return -1
    if arr[0] == target:
        return 0
    n, i = len(arr), 1
    while i < n and arr[i] <= target:
        i *= 2
    return binary_search(arr, i // 2, min(i, n - 1), target)

def binary_search(arr, low, high, target):
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target: return mid
        elif arr[mid] < target: low = mid + 1
        else: high = mid - 1
    return -1

arr = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]

```

```
print(exponential_search(arr, 70)) # 6
```

Time & Space Complexity

Complexity	Value
Best Case	O(1)
Average Case	O(log n)
Worst Case	O(log n)

2.6. Ternary Search

Theory

Ternary Search divides the sorted array into three parts using two midpoints and discards two-thirds of the search space at each step.

Algorithm

1. Compute $mid1 = low + (high - low) // 3$ and $mid2 = high - (high - low) // 3$.
2. If target equals $arr[mid1]$ or $arr[mid2]$, return that index.
3. Determine which third the target lies in and recurse.
4. Return -1 if not found.

Code (Python)

```
def ternary_search(arr, low, high, target):
    if high < low:
        return -1
    mid1 = low + (high - low) // 3
    mid2 = high - (high - low) // 3
    if arr[mid1] == target: return mid1
    if arr[mid2] == target: return mid2
    if target < arr[mid1]:
        return ternary_search(arr, low, mid1 - 1, target)
    elif target > arr[mid2]:
        return ternary_search(arr, mid2 + 1, high, target)
    else:
        return ternary_search(arr, mid1 + 1, mid2 - 1, target)

arr = list(range(1, 11))
print(ternary_search(arr, 0, len(arr)-1, 5)) # 4
```

Time & Space Complexity

Complexity	Value
Best Case	O(1)

Average Case	$O(\log^2 n)$
Worst Case	$O(\log^2 n)$

3. Dynamic Programming Algorithms

3.1. Fibonacci Sequence

Theory

The Fibonacci sequence (0, 1, 1, 2, 3, 5, 8...) is computed with DP by storing previously computed values to avoid redundant recursion.

Algorithm

1. Initialize $\text{fib}[0]=0$ and $\text{fib}[1]=1$.
2. For i from 2 to n : $\text{fib}[i] = \text{fib}[i-1] + \text{fib}[i-2]$.
3. Return $\text{fib}[n]$.

Code (Python)

```
def fibonacci(n):
    if n <= 1:
        return n
    fib = [0] * (n + 1)
    fib[1] = 1
    for i in range(2, n + 1):
        fib[i] = fib[i-1] + fib[i-2]
    return fib[n]

print(fibonacci(10)) # 55
```

Time & Space Complexity

Complexity	Value
Time Complexity	$O(n)$
Space Complexity	$O(n)$

3.2. 0/1 Knapsack Problem

Theory

Given items with weights and values, and a capacity W , find the maximum value achievable without exceeding W . Each item can be taken at most once.

Algorithm

1. Create a 2D DP table $\text{dp}[i][w]$.
2. For each item i and capacity w : either skip item or include it.
3. $\text{dp}[i][w] = \max(\text{dp}[i-1][w], \text{values}[i-1] + \text{dp}[i-1][w-\text{weights}[i-1]])$.

4. Return `dp[n][capacity]`.

Code (Python)

```
def knapsack(weights, values, capacity):
    n = len(values)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for w in range(1, capacity + 1):
            if weights[i-1] <= w:
                dp[i][w] = max(values[i-1] + dp[i-1][w - weights[i-1]],
                                dp[i-1][w])
            else:
                dp[i][w] = dp[i-1][w]
    return dp[n][capacity]

weights = [1, 3, 4, 5]
values = [1, 4, 5, 7]
capacity = 7
print(knapsack(weights, values, capacity)) # 9
```

Time & Space Complexity

Complexity	Value
Time Complexity	$O(n * \text{capacity})$
Space Complexity	$O(n * \text{capacity})$

3.3. Longest Common Subsequence (LCS)

Theory

LCS finds the longest subsequence present in both strings. Used in diff tools, bioinformatics, and version control.

Algorithm

1. Create `dp[i][j]` = LCS length of `X[:i]` and `Y[:j]`.
2. If `X[i-1] == Y[j-1]`: `dp[i][j] = dp[i-1][j-1] + 1`.
3. Else: `dp[i][j] = max(dp[i-1][j], dp[i][j-1])`.
4. Return `dp[m][n]`.

Code (Python)

```
def lcs(X, Y):
    m, n = len(X), len(Y)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if X[i-1] == Y[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
```

```

        else:
            dp[i][j] = max(dp[i-1][j], dp[i][j-1])
    return dp[m][n]

print(lcs("AGGTAB", "GXTXAYB")) # 4 (GTAB)

```

Time & Space Complexity

Complexity	Value
Time Complexity	$O(m * n)$
Space Complexity	$O(m * n)$

3.4. Longest Increasing Subsequence (LIS)

Theory

LIS finds the length of the longest subsequence where elements are strictly increasing. Used in patience sorting and scheduling.

Algorithm

1. Initialize $dp[i] = 1$ for all i .
2. For each i , check all $j < i$: if $arr[j] < arr[i]$, $dp[i] = \max(dp[i], dp[j]+1)$.
3. Return the maximum value in dp .

Code (Python)

```

def lis(arr):
    if not arr:
        return 0
    n = len(arr)
    dp = [1] * n
    for i in range(1, n):
        for j in range(i):
            if arr[j] < arr[i]:
                dp[i] = max(dp[i], dp[j] + 1)
    return max(dp)

arr = [10, 22, 9, 33, 21, 50, 41, 60]
print(lis(arr)) # 5

```

Time & Space Complexity

Complexity	Value
Time Complexity	$O(n^2)$
Space Complexity	$O(n)$

3.5. Matrix Chain Multiplication

Theory

Determines the optimal parenthesization of a chain of matrices to minimize scalar multiplications.

Algorithm

1. $dp[i][j]$ = min cost to multiply matrices i through j .
2. Try every split point k between i and j .
3. $cost = dp[i][k] + dp[k+1][j] + p[i-1]*p[k]*p[j]$.
4. Return $dp[1][n-1]$.

Code (Python)

```
def matrix_chain_order(p):
    n = len(p)
    dp = [[0] * n for _ in range(n)]
    for length in range(2, n):
        for i in range(1, n - length + 1):
            j = i + length - 1
            dp[i][j] = float('inf')
            for k in range(i, j):
                cost = dp[i][k] + dp[k+1][j] + p[i-1]*p[k]*p[j]
                dp[i][j] = min(dp[i][j], cost)
    return dp[1][n - 1]

print(matrix_chain_order([1, 2, 3, 4])) # 18
```

Time & Space Complexity

Complexity	Value
Time Complexity	$O(n^3)$
Space Complexity	$O(n^2)$

3.6. Edit Distance (Levenshtein)

Theory

Finds the minimum number of single-character edits (insert, delete, replace) needed to transform one string into another.

Algorithm

1. $dp[i][j]$ = edit distance between $s1[:i]$ and $s2[:j]$.
2. Base cases: $dp[i][0]=i$, $dp[0][j]=j$.
3. If $s1[i-1]==s2[j-1]$: $dp[i][j]=dp[i-1][j-1]$.

4. Else: $dp[i][j] = 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$.

Code (Python)

```
def edit_distance(s1, s2):
    m, n = len(s1), len(s2)
    dp = [[0]*(n+1) for _ in range(m+1)]
    for i in range(m+1): dp[i][0] = i
    for j in range(n+1): dp[0][j] = j
    for i in range(1, m+1):
        for j in range(1, n+1):
            if s1[i-1] == s2[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])
    return dp[m][n]

print(edit_distance("sunday", "saturday")) # 3
```

Time & Space Complexity

Complexity	Value
Time Complexity	$O(m * n)$
Space Complexity	$O(m * n)$

4. Graph Algorithms

4.1. Depth First Search (DFS)

Theory

DFS explores as far as possible along each branch before backtracking. Uses a stack (implicit via recursion). Used in cycle detection, topological sort, SCC.

Algorithm

1. Mark the starting node as visited.
2. Recursively visit all unvisited neighbours.
3. Backtrack when no unvisited neighbours remain.

Code (Python)

```
from collections import defaultdict

def dfs(graph, node, visited=None):
    if visited is None:
        visited = set()
    visited.add(node)
    print(node, end=' ')
    for neighbour in graph[node]:
        if neighbour not in visited:
            dfs(graph, neighbour, visited)

graph = defaultdict(list, {
    0: [1, 2], 1: [0, 3], 2: [0, 3],
    3: [1, 2, 4], 4: [3, 5], 5: [4]
})
dfs(graph, 0) # 0 1 3 2 4 5
```

Time & Space Complexity

Complexity	Value
Time Complexity	$O(V + E)$
Space Complexity	$O(V)$

4.2. Breadth First Search (BFS)

Theory

BFS explores all neighbours at the current depth before moving deeper. Uses a queue. Finds shortest paths in unweighted graphs.

Algorithm

1. Enqueue the start node and mark it visited.
2. Dequeue a node, process it, and enqueue its unvisited neighbours.
3. Repeat until the queue is empty.

Code (Python)

```
from collections import deque, defaultdict

def bfs(graph, start):
    visited = {start}
    queue = deque([start])
    while queue:
        node = queue.popleft()
        print(node, end=' ')
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)

graph = defaultdict(list, {
    0: [1, 2], 1: [0, 3], 2: [0, 3],
    3: [1, 2, 4], 4: [3, 5], 5: [4]
})
bfs(graph, 0) # 0 1 2 3 4 5
```

Time & Space Complexity

Complexity	Value
Time Complexity	$O(V + E)$
Space Complexity	$O(V)$

4.3. Dijkstra's Algorithm

Theory

Finds the shortest paths from a source to all other vertices in a graph with non-negative edge weights. Uses a min-heap priority queue.

Algorithm

1. Set $\text{dist}[\text{source}] = 0$ and all others to infinity.
2. Use a priority queue to always expand the nearest unvisited node.
3. Relax edges: if $\text{dist}[u] + w < \text{dist}[v]$, update $\text{dist}[v]$.
4. Repeat until all nodes are processed.

Code (Python)

```

import heapq
from collections import defaultdict

def dijkstra(graph, src, V):
    dist = [float('inf')] * V
    dist[src] = 0
    pq = [(0, src)] # (distance, vertex)
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                heapq.heappush(pq, (dist[v], v))
    return dist

graph = defaultdict(list)
graph[0] += [(1,9), (2,6), (3,5), (4,3)]
graph[2] += [(1,2), (3,4)]
print(dijkstra(graph, 0, 5))

```

Time & Space Complexity

Complexity	Value
Time Complexity	$O((V + E) \log V)$
Space Complexity	$O(V)$

4.4. Bellman-Ford Algorithm

Theory

Finds shortest paths from a source even with negative edge weights. Detects negative-weight cycles. Slower than Dijkstra but more general.

Algorithm

1. Initialize $\text{dist}[\text{src}] = 0$, all others = infinity.
2. Relax every edge $V-1$ times.
3. Run one more pass: if any edge still relaxes, a negative cycle exists.

Code (Python)

```

def bellman_ford(edges, V, src):
    dist = [float('inf')] * V
    dist[src] = 0
    for _ in range(V - 1):
        for u, v, w in edges:
            if dist[u] != float('inf') and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w

```



```

# Detect negative cycle
for u, v, w in edges:
    if dist[u] != float('inf') and dist[u] + w < dist[v]:
        print("Negative weight cycle detected")
        return None
return dist

edges = [(0,1, -1), (0,2,4), (1,2,3), (1,3,2), (1,4,2),
        (3,2,5), (3,1,1), (4,3, -3)]
print(bellman_ford(edges, 5, 0))

```

Time & Space Complexity

Complexity	Value
Time Complexity	$O(V * E)$
Space Complexity	$O(V)$

4.5. Floyd-Warshall Algorithm

Theory

Finds shortest paths between ALL pairs of vertices. Works with negative weights but not negative cycles. Uses dynamic programming on the vertex set.

Algorithm

1. Initialize $\text{dist}[i][j]$ = direct edge weight (inf if no edge).
2. For each intermediate vertex k , update $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][k] + \text{dist}[k][j])$.
3. Print the final matrix.

Code (Python)

```

INF = float('inf')

def floyd_warshall(graph):
    V = len(graph)
    dist = [row[:] for row in graph] # deep copy
    for k in range(V):
        for i in range(V):
            for j in range(V):
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
    return dist

graph = [
    [0, 3, INF, 7],
    [8, 0, 2, INF],
    [5, INF, 0, 1],
    [2, INF, INF, 0]
]

```

```

result = floyd_warshall(graph)
for row in result:
    print(row)

```

Time & Space Complexity

Complexity	Value
Time Complexity	$O(V^3)$
Space Complexity	$O(V^2)$

4.6. Kruskal's Algorithm (MST)

Theory

Greedy algorithm that builds the Minimum Spanning Tree by sorting all edges and adding the lightest edge that doesn't form a cycle, using Union-Find.

Algorithm

1. Sort all edges by weight.
2. For each edge (u, v, w): if u and v are in different components, add the edge.
3. Use Union-Find (path compression + union by rank) to track components.
4. Stop after V-1 edges are added.

Code (Python)

```

def kruskal(V, edges):
    edges.sort(key=lambda e: e[2])
    parent = list(range(V))
    rank = [0] * V

    def find(x):
        if parent[x] != x:
            parent[x] = find(parent[x])
        return parent[x]

    def union(x, y):
        px, py = find(x), find(y)
        if px == py: return False
        if rank[px] < rank[py]: px, py = py, px
        parent[py] = px
        if rank[px] == rank[py]: rank[px] += 1
        return True

    mst, cost = [], 0
    for u, v, w in edges:
        if union(u, v):
            mst.append((u, v, w))
            cost += w

```

```

    return mst, cost

edges = [(0,1,10),(0,2,6),(0,3,5),(1,3,15),(2,3,4)]
mst, cost = kruskal(4, edges)
print("MST:", mst, " Total cost:", cost)

```

Time & Space Complexity

Complexity	Value
Time Complexity	$O(E \log E + V \log V)$
Space Complexity	$O(V)$

4.7. Prim's Algorithm (MST)

Theory

Greedy algorithm that grows the MST from a seed vertex by always picking the cheapest edge connecting the growing MST to an outside vertex.

Algorithm

1. Start with vertex 0 in the MST.
2. At each step, add the cheapest edge leaving the current MST.
3. Use a min-heap for efficient edge selection.
4. Repeat until all V vertices are included.

Code (Python)

```

import heapq

def prim(graph, V):
    in_mst = [False] * V
    key = [float('inf')] * V
    parent = [-1] * V
    key[0] = 0
    pq = [(0, 0)] # (weight, vertex)
    while pq:
        w, u = heapq.heappop(pq)
        if in_mst[u]: continue
        in_mst[u] = True
        for v, weight in graph[u]:
            if not in_mst[v] and weight < key[v]:
                key[v] = weight
                parent[v] = u
                heapq.heappush(pq, (weight, v))
    return [(parent[i], i, key[i]) for i in range(1, V)]

graph = {0:[(1,2),(3,6)], 1:[(0,2),(2,3),(3,8),(4,5)],
        2:[(1,3),(4,7)], 3:[(0,6),(1,8),(4,9)], 4:[(1,5),(2,7),(3,9)]}

```

```
print(prim(graph, 5))
```

Time & Space Complexity

Complexity	Value
Time Complexity	$O((V + E) \log V)$
Space Complexity	$O(V)$

4.8. A* Search Algorithm

Theory

A* is a best-first pathfinding algorithm that uses a heuristic $h(n)$ to guide its search. It combines Dijkstra's accuracy with Greedy Best-First speed by evaluating $f(n) = g(n) + h(n)$, where $g(n)$ is the cost so far and $h(n)$ is the estimated cost to goal.

Algorithm

1. Maintain an open list (priority queue) and a closed set.
2. For each node, compute $f(n) = g(n) + h(n)$; use Manhattan distance as heuristic.
3. Always expand the node with the lowest $f(n)$ from the open list.
4. When the goal is reached, reconstruct the path via parent pointers.
5. Repeat until goal found or open list is empty.

Code (Python)

```
import heapq

def astar(grid, start, goal):
    rows, cols = len(grid), len(grid[0])
    def h(a, b): return abs(a[0]-b[0]) + abs(a[1]-b[1]) # Manhattan

    open_list = [(0 + h(start, goal), 0, start, [])]
    visited = set()

    while open_list:
        f, g, current, path = heapq.heappop(open_list)
        if current in visited:
            continue
        visited.add(current)
        path = path + [current]
        if current == goal:
            return path
        r, c = current
        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
            nr, nc = r+dr, c+dc
            if 0<=nr<rows and 0<=nc<cols and grid[nr][nc]==0:
                nb = (nr, nc)
```

```

        if nb not in visited:
            ng = g + 1
            heapq.heappush(open_list, (ng+h(nb,goal), ng, nb, path))

    return None

# 0=walkable, 1=wall
grid = [[0,0,0],[0,1,0],[0,0,0]]
print(astar(grid, (0,0), (2,2)))
# [(0,0),(1,0),(2,0),(2,1),(2,2)]

```

Time & Space Complexity

Complexity	Value
Time Complexity	$O(b^d)$ — b =branching factor, d =depth
Space Complexity	$O(b^d)$

4.9. Tarjan's Algorithm (SCC)

Theory

Finds all Strongly Connected Components (SCCs) in a directed graph using a single DFS pass. Uses discovery times and low-link values to identify SCC roots.

Algorithm

1. Run DFS, tracking discovery time and low-link value for each node.
2. Push each visited node onto a stack.
3. If a node's low-link equals its discovery time, it is an SCC root.
4. Pop nodes from the stack until the root is reached — they form one SCC.

Code (Python)

```

from collections import defaultdict

def tarjan_scc(graph, V):
    disc = [-1] * V
    low = [-1] * V
    on_stack = [False] * V
    stack = []
    sccs = []
    timer = [0]

    def dfs(u):
        disc[u] = low[u] = timer[0]
        timer[0] += 1
        stack.append(u)
        on_stack[u] = True
        for v in graph[u]:
            if disc[v] == -1:

```

```

        dfs(v)
        low[u] = min(low[u], low[v])
    elif on_stack[v]:
        low[u] = min(low[u], disc[v])
    if low[u] == disc[u]: # u is root of an SCC
        scc = []
        while True:
            w = stack.pop()
            on_stack[w] = False
            scc.append(w)
            if w == u: break
        sccs.append(scc)

    for i in range(V):
        if disc[i] == -1:
            dfs(i)
    return sccs

graph = defaultdict(list, {0:[1], 1:[2], 2:[0], 3:[1,4], 4:[5], 5:[3]})
print(tarjan_scc(graph, 6))
# [[2,1,0],[5,4,3]] (SCCs)

```

Time & Space Complexity

Complexity	Value
Time Complexity	$O(V + E)$
Space Complexity	$O(V)$

4.10. Kosaraju's Algorithm (SCC)

Theory

Finds SCCs using two DFS passes: one on the original graph (to get finish order) and one on the reversed graph (to collect SCCs).

Algorithm

1. Run DFS on the original graph; push nodes to a stack in order of finish time.
2. Reverse all edges in the graph.
3. Pop nodes from the stack and run DFS on the reversed graph.
4. Each DFS tree in the second pass is one SCC.

Code (Python)

```

from collections import defaultdict

def kosaraju_scc(graph, V):
    visited = [False] * V
    stack = []

```

```

def dfs1(u):
    visited[u] = True
    for v in graph[u]:
        if not visited[v]: dfs1(v)
    stack.append(u)

for i in range(V):
    if not visited[i]: dfs1(i)

# Build reversed graph
rev = defaultdict(list)
for u in graph:
    for v in graph[u]:
        rev[v].append(u)

visited = [False] * V
sccs = []

def dfs2(u, comp):
    visited[u] = True
    comp.append(u)
    for v in rev[u]:
        if not visited[v]: dfs2(v, comp)

while stack:
    u = stack.pop()
    if not visited[u]:
        comp = []
        dfs2(u, comp)
        sccs.append(comp)
return sccs

graph = defaultdict(list, {0:[1], 1:[2], 2:[0], 3:[1,4], 4:[5], 5:[3]})
print(kosaraju_scc(graph, 6))
# [[0,2,1],[3,5,4]] (SCCs)

```

Time & Space Complexity

Complexity	Value
Time Complexity	$O(V + E)$
Space Complexity	$O(V + E)$

4.11. Cycle Detection (Directed & Undirected)

Theory

Detects whether a graph contains a cycle. For directed graphs, DFS uses a recursion stack. For undirected graphs, DFS checks if a visited neighbour is not the parent of the current node.

Algorithm

1. Directed: use DFS with a 'rec_stack' to track nodes in the current path.
2. If a neighbour is already in rec_stack, a cycle exists.
3. Undirected: use DFS tracking the parent node.
4. If a visited neighbour is not the parent, a cycle exists.

Code (Python)

```
from collections import defaultdict

# --- Directed graph cycle detection ---
def has_cycle_directed(graph, V):
    visited = [False] * V
    rec_stack = [False] * V

    def dfs(u):
        visited[u] = rec_stack[u] = True
        for v in graph[u]:
            if not visited[v] and dfs(v): return True
            if rec_stack[v]: return True
        rec_stack[u] = False
        return False

    return any(not visited[i] and dfs(i) for i in range(V))

# --- Undirected graph cycle detection ---
def has_cycle_undirected(graph, V):
    visited = [False] * V

    def dfs(u, parent):
        visited[u] = True
        for v in graph[u]:
            if not visited[v]:
                if dfs(v, u): return True
            elif v != parent:
                return True
        return False

    return any(not visited[i] and dfs(i, -1) for i in range(V))

dg = defaultdict(list, {0:[1], 1:[2], 2:[0]}) # directed cycle
ug = defaultdict(list, {0:[1], 1:[0,2], 2:[1,3], 3:[2,0]}) # undirected cycle
print(has_cycle_directed(dg, 3)) # True
print(has_cycle_undirected(ug, 4)) # True
```

Time & Space Complexity

Complexity	Value
Time Complexity	$O(V + E)$
Space Complexity	$O(V)$

4.12. Topological Sorting

Theory

An ordering of vertices in a DAG such that for every directed edge $u \rightarrow v$, u comes before v . Used in task scheduling, build systems, and dependency resolution.

Algorithm

1. Run DFS on each unvisited node.
2. After fully exploring a node, push it onto a stack.
3. The topological order is the reverse of the finish order (pop from stack).

Code (Python)

```
from collections import defaultdict

def topological_sort(graph, V):
    visited = [False] * V
    stack = []

    def dfs(u):
        visited[u] = True
        for v in graph[u]:
            if not visited[v]:
                dfs(v)
        stack.append(u)

    for i in range(V):
        if not visited[i]:
            dfs(i)

    return stack[::-1] # reverse = topological order

graph = defaultdict(list, {5:[2,0], 4:[0,1], 2:[3], 3:[1]})
print(topological_sort(graph, 6))
# [5, 4, 2, 3, 1, 0] (one valid ordering)
```

Time & Space Complexity

Complexity	Value
Time Complexity	$O(V + E)$
Space Complexity	$O(V)$

4.13. Union-Find (Disjoint Set Union)

Theory

A data structure that tracks elements partitioned into disjoint sets. Supports near-constant time Find and Union operations using path compression and union by rank. Used in Kruskal's MST and cycle detection.

Algorithm

1. Find: returns the root of the set containing x (with path compression).
2. Union: merges two sets using union by rank to keep trees flat.
3. If $\text{Find}(x) == \text{Find}(y)$, x and y are in the same set (cycle detected).

Code (Python)

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x]) # path compression
        return self.parent[x]

    def union(self, x, y):
        px, py = self.find(x), self.find(y)
        if px == py:
            return False # already in same set (cycle!)
        # union by rank
        if self.rank[px] < self.rank[py]:
            px, py = py, px
        self.parent[py] = px
        if self.rank[px] == self.rank[py]:
            self.rank[px] += 1
        return True

uf = UnionFind(5)
uf.union(0, 1)
uf.union(1, 2)
print(uf.find(0) == uf.find(2)) # True — same set
print(uf.union(0, 2)) # False — already connected (cycle)
uf.union(3, 4)
print(uf.find(0) == uf.find(3)) # False — different sets
```

Time & Space Complexity

Complexity	Value
Find / Union	$O(\alpha(n))$ — nearly $O(1)$ amortized
Space Complexity	$O(n)$

5. Greedy Algorithms

5.1. Activity Selection Problem

Theory

Given n activities with start and finish times, select the maximum number of non-overlapping activities.
Classic greedy: always pick the activity that ends earliest.

Algorithm

1. Sort activities by finish time.
2. Select the first activity.
3. For each subsequent activity, select it if its start time $\geq$ last finish time.

Code (Python)

```
def activity_selection(activities):
    # activities: list of (start, finish)
    activities.sort(key=lambda a: a[1])
    selected = [activities[0]]
    last_finish = activities[0][1]
    for start, finish in activities[1:]:
        if start >= last_finish:
            selected.append((start, finish))
            last_finish = finish
    return selected

acts = [(1,2), (3,4), (0,6), (5,7), (8,9), (5,9)]
print(activity_selection(acts))
# [(1,2), (3,4), (5,7), (8,9)]
```

Time & Space Complexity

Complexity	Value
Time Complexity	$O(n \log n)$ — dominated by sort
Space Complexity	$O(1)$

6. Backtracking Algorithms

6.1. N-Queens Problem

Theory

Place N queens on an N×N chessboard so that no two queens threaten each other. Uses backtracking to explore placements row by row.

Algorithm

1. Place a queen in each row, trying each column.
2. Check if the placement is safe (no conflicts on column or diagonals).
3. Recurse to the next row; backtrack if no safe column exists.

Code (Python)

```
def solve_n_queens(n):
    solutions = []
    def backtrack(row, cols, diag1, diag2, board):
        if row == n:
            solutions.append([''.join(r) for r in board])
            return
        for col in range(n):
            if col in cols or (row-col) in diag1 or (row+col) in diag2:
                continue
            board[row][col] = 'Q'
            backtrack(row+1, cols|{col}, diag1|{row-col},
                    diag2|{row+col}, board)
            board[row][col] = '.'
        board = [['.']*n for _ in range(n)]
        backtrack(0, set(), set(), set(), board)
    return solutions

print(f"4-Queens solutions: {len(solve_n_queens(4))}") # 2
```

Time & Space Complexity

Complexity	Value
Time Complexity	O(n!)
Space Complexity	O(n)

6.2. Sudoku Solver

Theory

Fills a 9×9 Sudoku grid by trying digits 1-9 in each empty cell and backtracking when a conflict is detected.

Algorithm

1. Find the next empty cell.
2. Try digits 1-9; check row, column, and 3×3 box constraints.
3. Recurse; if a conflict is found later, backtrack and try the next digit.

Code (Python)

```
def solve_sudoku(board):
    empty = find_empty(board)
    if not empty:
        return True # solved!
    row, col = empty
    for num in map(str, range(1, 10)):
        if is_valid(board, row, col, num):
            board[row][col] = num
            if solve_sudoku(board):
                return True
            board[row][col] = '.' # backtrack
    return False

def find_empty(board):
    for r in range(9):
        for c in range(9):
            if board[r][c] == '.':
                return r, c

def is_valid(board, row, col, num):
    if num in board[row]: return False
    if num in [board[r][col] for r in range(9)]: return False
    br, bc = 3*(row//3), 3*(col//3)
    for r in range(br, br+3):
        for c in range(bc, bc+3):
            if board[r][c] == num: return False
    return True
```

Time & Space Complexity

Complexity	Value
Time Complexity	$O(9^{(n^2)})$ worst case
Space Complexity	$O(n^2)$

7. Mathematical Algorithms

7.1. Euclidean Algorithm (GCD)

Theory

Efficiently computes the Greatest Common Divisor (GCD) of two integers using the property: $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$.

Algorithm

1. If $b == 0$, return a .
2. Otherwise, return $\text{gcd}(b, a \bmod b)$.

Code (Python)

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

print(gcd(48, 18)) # 6
print(gcd(100, 75)) # 25

# Python also provides math.gcd():
import math
print(math.gcd(48, 18)) # 6
```

Time & Space Complexity

Complexity	Value
Time Complexity	$O(\log(\min(a, b)))$
Space Complexity	$O(1)$

7.2. Sieve of Eratosthenes

Theory

Finds all prime numbers up to n by iteratively marking multiples of each prime as composite. One of the most efficient primality algorithms.

Algorithm

1. Create a boolean array `is_prime[0..n]`, initialized True.
2. Mark 0 and 1 as not prime.
3. For each p from 2 to $\sqrt{n}$: if `is_prime[p]`, mark all multiples of p as False.

4. Collect all indices still True.

Code (Python)

```
def sieve_of_eratosthenes(n):
    is_prime = [True] * (n + 1)
    is_prime[0] = is_prime[1] = False
    for p in range(2, int(n**0.5) + 1):
        if is_prime[p]:
            for multiple in range(p*p, n+1, p):
                is_prime[multiple] = False
    return [p for p, prime in enumerate(is_prime) if prime]

print(sieve_of_eratosthenes(30))
# [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

Time & Space Complexity

Complexity	Value
Time Complexity	$O(n \log \log n)$
Space Complexity	$O(n)$

7.3. Modular Exponentiation

Theory

Computes $(\text{base}^{\text{exp}}) \% \text{mod}$ efficiently using repeated squaring, avoiding huge intermediate numbers. Essential in cryptography.

Algorithm

1. If $\text{exp} == 0$, return 1.
2. If exp is even: $\text{result} = (\text{base}^2 \% \text{mod})^{(\text{exp}/2)} \% \text{mod}$.
3. If exp is odd: $\text{result} = \text{base} * (\text{base}^{(\text{exp}-1)} \% \text{mod}) \% \text{mod}$.

Code (Python)

```
def mod_exp(base, exp, mod):
    result = 1
    base %= mod
    while exp > 0:
        if exp % 2 == 1:
            result = (result * base) % mod
        exp //= 2
        base = (base * base) % mod
    return result

# Python built-in: pow(base, exp, mod) – same algorithm
print(mod_exp(2, 10, 1000)) # 24
print(pow(2, 10, 1000)) # 24
```

Time & Space Complexity

Complexity	Value
Time Complexity	$O(\log \exp)$
Space Complexity	$O(1)$

8. Bit Manipulation Algorithms

8.1. Counting Set Bits

Theory

Counts the number of bits set to 1 in the binary representation of a number. Also known as the Hamming Weight or population count. Python's `bin()` and `bin().count('1')` offer a concise one-liner.

Algorithm

1. Iterate through each bit of the number.
2. Use bitwise AND with 1 to check if the least significant bit is set.
3. Right-shift the number by 1 and repeat until it becomes 0.

Code (Python)

```
def count_set_bits(n):
    count = 0
    while n:
        count += n & 1
        n >>= 1
    return count

# Brian Kernighan's faster trick: n & (n-1) clears the lowest set bit
def count_set_bits_bk(n):
    count = 0
    while n:
        n &= (n - 1)
        count += 1
    return count

# Python one-liner
def count_set_bits_py(n):
    return bin(n).count('1')

n = 29 # binary: 11101 → 4 set bits
print(count_set_bits(n)) # 4
print(count_set_bits_bk(n)) # 4
print(count_set_bits_py(n)) # 4
```

Time & Space Complexity

Complexity	Value
Time Complexity	$O(\log n)$ — number of bits
Space Complexity	$O(1)$

8.2. Finding the Only Non-Duplicated Element

Theory

Given an array where every element appears exactly twice except one, find the unique element in $O(n)$ time and $O(1)$ space using the XOR property: $a \text{ XOR } a = 0$ and $a \text{ XOR } 0 = a$.

Algorithm

1. Initialize result = 0.
2. XOR every element in the array with result.
3. Pairs cancel out ($x \text{ XOR } x = 0$); the lone element remains.

Code (Python)

```
def find_single(nums):
    result = 0
    for num in nums:
        result ^= num
    return result

# Example 1: one unique element
print(find_single([1, 2, 3, 2, 1])) # 3

# Example 2: larger array
print(find_single([4, 1, 2, 1, 2])) # 4

# Python functional one-liner using functools.reduce
from functools import reduce
from operator import xor
print(reduce(xor, [1, 2, 3, 2, 1])) # 3
```

Time & Space Complexity

Complexity	Value
Time Complexity	$O(n)$
Space Complexity	$O(1)$

9. Tree Algorithms

9.1. Tree Traversals

Theory

Three fundamental DFS traversals of a binary tree, each visiting the root at a different point relative to its subtrees.

Algorithm

1. In-order ($L \rightarrow \text{Root} \rightarrow R$): gives sorted output for a BST.
2. Pre-order ($\text{Root} \rightarrow L \rightarrow R$): used for tree copying / serialization.
3. Post-order ($L \rightarrow R \rightarrow \text{Root}$): used for tree deletion / evaluation.

Code (Python)

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def inorder(root):
    return inorder(root.left) + [root.val] + inorder(root.right) if root else []

def preorder(root):
    return [root.val] + preorder(root.left) + preorder(root.right) if root else []

def postorder(root):
    return postorder(root.left) + postorder(root.right) + [root.val] if root else []

# 1
# / \
# 2 3
root = TreeNode(1, TreeNode(2), TreeNode(3))
print("In-order: ", inorder(root)) # [2, 1, 3]
print("Pre-order: ", preorder(root)) # [1, 2, 3]
print("Post-order:", postorder(root)) # [2, 3, 1]
```

Time & Space Complexity

Complexity	Value
Time Complexity	$O(n)$
Space Complexity	$O(h)$ — h = tree height

9.2. Binary Search Tree (BST)

Theory

A BST stores elements so that for every node, left subtree values are smaller and right subtree values are larger. Supports $O(\log n)$ operations on average.

Algorithm

1. Insert: compare with current node; go left if smaller, right if larger.
2. Search: same comparison logic as insert.
3. Delete: three cases — leaf, one child, two children (replace with in-order successor).

Code (Python)

```
class BST:
    class Node:
        def __init__(self, val):
            self.val, self.left, self.right = val, None, None

    def __init__(self): self.root = None

    def insert(self, val):
        def _ins(node, val):
            if not node: return self.Node(val)
            if val < node.val: node.left = _ins(node.left, val)
            else: node.right = _ins(node.right, val)
            return node
        self.root = _ins(self.root, val)

    def search(self, val):
        node = self.root
        while node:
            if val == node.val: return True
            node = node.left if val < node.val else node.right
        return False

bst = BST()
for v in [5, 3, 7, 1, 4]:
    bst.insert(v)
print(bst.search(4)) # True
print(bst.search(6)) # False
```

Time & Space Complexity

Complexity	Value
Search/Insert	$O(\log n)$ avg, $O(n)$ worst
Space Complexity	$O(n)$

9.3. Lowest Common Ancestor (LCA)

Theory

Finds the deepest node that is an ancestor of both given nodes p and q in a binary tree.

Algorithm

1. If root is None or equals p or q, return root.
2. Recursively search left and right subtrees.
3. If both return non-None, root is the LCA.
4. Otherwise, return the non-None result.

Code (Python)

```
def lca(root, p, q):
    if not root or root == p or root == q:
        return root
    left = lca(root.left, p, q)
    right = lca(root.right, p, q)
    if left and right:
        return root # p and q in different subtrees
    return left or right

# 3
# / \
# 5 1
# / \
# 6 2
n3,n5,n1,n6,n2 = (TreeNode(v) for v in [3,5,1,6,2])
n3.left,n3.right,n5.left,n5.right = n5,n1,n6,n2
print(lca(n3, n5, n1).val) # 3
print(lca(n3, n5, n6).val) # 5
```

Time & Space Complexity

Complexity	Value
Time Complexity	O(n)
Space Complexity	O(h) — recursion stack

9.4. AVL Tree (Self-Balancing BST)

Theory

An AVL tree maintains $|\text{height}(\text{left}) - \text{height}(\text{right})| \leq 1$ at every node via single and double rotations after insertions, guaranteeing $O(\log n)$ operations.

Algorithm

1. Insert as in a regular BST.
2. Update heights; compute balance factor = $\text{height}(\text{left}) - \text{height}(\text{right})$.
3. Apply LL, RR, LR, or RL rotation if $|\text{balance}| > 1$.
4. All operations remain $O(\log n)$ guaranteed.

Code (Python)

```
class AVLNode:
    def __init__(self, val):
        self.val, self.left, self.right, self.height = val, None, None, 1

def height(n): return n.height if n else 0
def balance(n): return height(n.left) - height(n.right) if n else 0

def right_rotate(y):
    x = y.left; T2 = x.right
    x.right = y; y.left = T2
    y.height = 1 + max(height(y.left), height(y.right))
    x.height = 1 + max(height(x.left), height(x.right))
    return x

def left_rotate(x):
    y = x.right; T2 = y.left
    y.left = x; x.right = T2
    x.height = 1 + max(height(x.left), height(x.right))
    y.height = 1 + max(height(y.left), height(y.right))
    return y

def avl_insert(node, val):
    if not node: return AVLNode(val)
    if val < node.val: node.left = avl_insert(node.left, val)
    else: node.right = avl_insert(node.right, val)
    node.height = 1 + max(height(node.left), height(node.right))
    bf = balance(node)
    if bf > 1:
        if val > node.left.val: node.left = left_rotate(node.left)
        return right_rotate(node)
    if bf < -1:
        if val < node.right.val: node.right = right_rotate(node.right)
        return left_rotate(node)
    return node

root = None
for v in [10, 20, 30, 40, 50, 25]:
    root = avl_insert(root, v)
print("Root:", root.val) # 30 (balanced)
```

Time & Space Complexity

Complexity	Value
Insert/Search/Delete	$O(\log n)$ guaranteed
Space Complexity	$O(n)$